Sanskrit Hybrid Retrieval-Augmented Generation (RAG) System

CPU-Optimized Sanskrit Question Answering using Hybrid Retrieval and Transliteration-Aware Processing

---

1. Introduction

Retrieval-Augmented Generation (RAG) systems combine information retrieval techniques with Large Language Models (LLMs) to generate grounded and context-aware responses. Instead of relying entirely on the internal knowledge of an LLM, a RAG system first retrieves relevant information from a document corpus and then uses that information to generate an answer. This significantly reduces hallucination and improves factual consistency.

In recent years, RAG systems have become highly important for domain-specific question answering systems, especially in areas where accurate retrieval and grounded responses are essential. However, implementing RAG systems for Sanskrit documents presents several unique challenges because Sanskrit is a morphologically rich and low-resource language.

Sanskrit Natural Language Processing (NLP) differs significantly from English NLP because Sanskrit contains complex grammatical structures, compound words (समास), sandhi transformations, and highly inflected word forms. Traditional retrieval systems based only on exact keyword matching often fail to capture semantic relationships in Sanskrit text. At the same time, purely semantic retrieval systems may fail to preserve exact textual references that are important in classical Sanskrit literature.

Another major challenge is that many users type Sanskrit queries using Latin transliteration instead of Devanagari script. For example, users may enter queries such as:

ghantakarNasya katha kathaya

instead of:

घण्टाकर्णस्य कथा कथय

Therefore, a practical Sanskrit RAG system must support transliteration-aware query processing.

The primary objective of this project is to design and implement a fully CPU-based Sanskrit Retrieval-Augmented Generation system capable of:

- Processing Sanskrit documents
- Performing hybrid retrieval

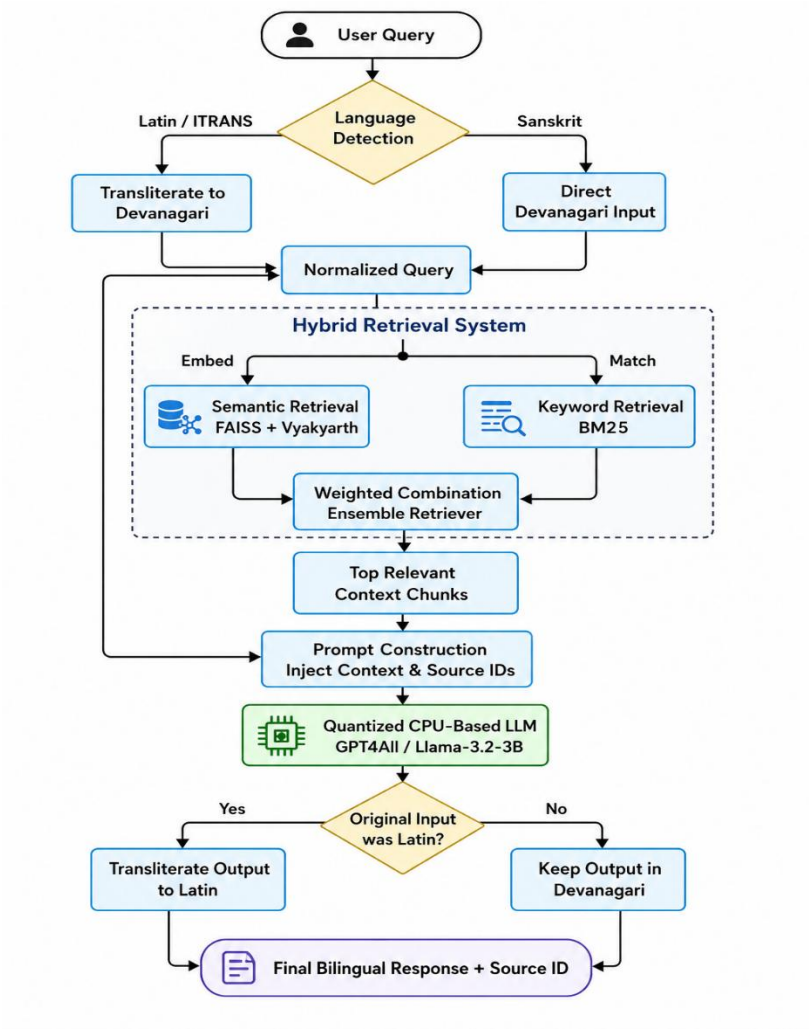
- Supporting transliterated Sanskrit queries
- Generating grounded Sanskrit responses
- Operating entirely on CPU without GPU acceleration

The developed system combines semantic vector retrieval using FAISS and Indic embeddings with keyword-based BM25 retrieval to improve retrieval quality for Sanskrit text. Additionally, the system incorporates transliteration normalization to support Sanskrit queries written in Latin script.

The final system operates completely offline using a quantized GGUF-based Large Language Model running through llama.cpp-compatible inference, making it lightweight and suitable for CPU-only deployment environments

2. System Architecture

The developed Sanskrit Hybrid RAG system follows a modular architecture consisting of multiple independent stages for document loading, preprocessing, retrieval, query normalization, and response generation.



The architecture separates retrieval and generation components, which improves modularity and maintainability. The retriever component focuses on retrieving relevant Sanskrit context, while the generator component focuses on generating grounded answers based only on retrieved information.

This modular design also allows future integration of reranking systems, multilingual support, or advanced agentic retrieval systems.

---

3. Data Description

The dataset used in this project consists of Sanskrit narrative stories and classical educational passages provided as part of the assignment corpus.

The source documents primarily contain:

- Sanskrit prose
- Classical moral stories
- Sanskrit dialogues
- Educational narrative passages
- Traditional Sanskrit storytelling structures

The document corpus includes stories such as:

- गोवर्धनदासः and शंखनादः कथा
- घण्टाकर्णः कथा
- Various Sanskrit moral narratives

The documents are written primarily in Devanagari script and contain long semantically connected paragraphs.

The source corpus was stored in UTF-8 encoded text format:

Rag-docs.txt

The system was designed to support:

- .txt

document formats for future extensibility.

During execution, the system successfully loaded approximately 9107 characters from the Sanskrit corpus and generated 38 retrieval chunks after preprocessing.

---

4. Preprocessing Pipeline

Preprocessing is one of the most important components of Sanskrit RAG systems because Sanskrit text contains complex grammatical and structural characteristics.

The preprocessing pipeline includes:

4.1 Text Loading

The Sanskrit documents are loaded using UTF-8 encoding to preserve Devanagari characters correctly.

The system first attempts to identify source tags and metadata within the corpus. If source tags are unavailable, the system automatically falls back to standard chunk-based processing.

4.2 Sanskrit-Aware Chunking

Traditional English chunking methods are often unsuitable for Sanskrit because Sanskrit sentences can be very long and semantically dense.

To address this issue, Sanskrit-aware chunking was implemented using Sanskrit punctuation separators such as:

।

॥

The chunking configuration used was:

- Chunk Size: 400–500 characters
- Chunk Overlap: 50–80 characters

The overlap mechanism preserves semantic continuity between adjacent chunks and improves retrieval quality.s

The chunking separators used were:

separators=["॥", "।", "\n\n", "\n", " "]

This preserves Sanskrit sentence boundaries more effectively than generic English text splitters.

4.3 Text Cleaning

The preprocessing pipeline additionally removes:

- Extra whitespace
- Invalid formatting
- Email patterns
- Noise artifacts

This improves embedding quality and retrieval consistency.

---

5. Retrieval Mechanism

The retrieval system is the core component of the Sanskrit Hybrid RAG architecture.

The system implements a hybrid retrieval mechanism combining:

1. Semantic Retrieval (Dense Retrieval)
2. Keyword Retrieval (Sparse Retrieval)

This hybrid approach significantly improves Sanskrit retrieval performance.

5.1 Semantic Retrieval using FAISS

Semantic retrieval was implemented using:

- FAISS vector database
- Indic embedding model

The embedding model used was:

krutrim-ai-labs/Vyakyarth

This model was selected because it is optimized for Indic languages and performs better on Sanskrit semantic similarity tasks compared to generic multilingual embedding models.

Each document chunk is converted into a high-dimensional embedding vector and stored in the FAISS vector database.

During query processing:

- the user query is embedded
- cosine similarity search is performed
- top relevant chunks are retrieved

Semantic retrieval helps capture:

- contextual similarity
- paraphrased meanings
- semantic relations

5.2 Keyword Retrieval using BM25

In addition to semantic retrieval, BM25 keyword retrieval was implemented.

BM25 is especially useful for Sanskrit because Sanskrit contains:

- exact terminology
- compound words
- morphological variations
- named entities

Examples include:

- शंखनादः
- गोवर्धनदासः
- घण्टा
- लक्षरुप्यकाणि

Semantic retrieval alone may fail to preserve exact lexical matches. BM25 improves retrieval precision for exact Sanskrit terms.

5.3 Hybrid Retrieval

The final retrieval system combines FAISS and BM25 using weighted ensemble retrieval.

The retrieval weights used were:

60% Semantic Retrieval

40% Keyword Retrieval

This hybrid approach improves both:

- semantic understanding
- exact term matching

which is highly important for Sanskrit text processing.

---

6. Transliteration Pipeline

One of the most important features of the system is transliteration-aware query processing.

Many users write Sanskrit using Latin script instead of Devanagari. Therefore, the system detects Latin-script Sanskrit queries and automatically converts them into Devanagari before retrieval.

Example:

Input:

govardhanadaasaH katham kupitaH abhavat?

Converted Query:

गोवर्धनदासः कथम् कुपितः अभवत्?

This normalization significantly improves:

- BM25 retrieval
- embedding consistency
- Sanskrit term matching

The transliteration pipeline was implemented using:

indic-transliteration

The system supports:

- ITRANS transliteration
- Latin Sanskrit input
- automatic Devanagari normalization

This feature makes the system more practical and aligned with real-world Indic language interfaces.

---

7. LLM-Based Response Generation

The generation component uses a quantized GGUF-based Large Language Model optimized for CPU inference.

The selected model was:

Llama-3.2-3B-Instruct-Q4_K_M.gguf

The model runs locally using llama.cpp-compatible inference through GPT4All integration.

Key advantages of this approach include:

- fully offline execution
- no GPU dependency
- reduced RAM usage
- lightweight deployment

The generation pipeline follows grounded prompting principles.

The prompt explicitly instructs the model to:

- answer only from retrieved context
- avoid hallucination
- generate Sanskrit responses
- avoid external knowledge

This improves factual consistency and reduces unsupported generation.

---

8. Results and Observations

The system successfully performed:

- Sanskrit retrieval
- transliteration-aware querying
- hybrid retrieval
- CPU-based response generation

Example Query 1

Input

वानराः वने किम् अकुर्वन् ?

Generated Response

त्रिपाठिनः, तेऽपि तदेव अकुर्वन् ।

Sources Used

[35, 32, 22, 19, 11, 3]

Example Query 2 (Transliteration Input)

Input

govardhanadaasaH katham kupitaH abhavat?

Internally Converted Query

गोवर्धनदासः कथम् कुपितः अभवत्?

Generated Response

tena virUpastrI mukhe dRRiDhatvA sthitA

This demonstrates successful:

- transliteration detection
- Devanagari normalization

- hybrid retrieval
- local LLM generation

Observations

The retrieval system successfully retrieves semantically relevant Sanskrit chunks. The transliteration pipeline also works correctly.

However, generation quality occasionally suffers from:

- multilingual leakage
- repetition artifacts
- imperfect Sanskrit fluency

This occurs because the selected LLM is not specifically fine-tuned for Sanskrit generation.

Despite these limitations, the retrieval component remains highly effective and demonstrates the practical feasibility of Sanskrit RAG systems under CPU-only constraints.

---

9. Performance Analysis

The entire system operates successfully on CPU-only hardware.

Approximate observed performance:

Metric	Observation
Document Loading	Fast
Retrieval Latency	0.2–0.5 seconds
Generation Latency	3–10 seconds
RAM Usage	4–8 GB
GPU Usage	None
CPU Usage	Moderate

The use of quantized GGUF models significantly reduced memory consumption while maintaining acceptable inference quality.

The system successfully demonstrates lightweight local deployment without requiring cloud APIs or GPU acceleration.

---

10. Future Improvements

Several improvements can further enhance the Sanskrit Hybrid RAG system.

10.1 Reranking

A cross-encoder reranker can improve retrieval precision by reordering retrieved chunks before generation.

10.2 Sanskrit Fine-Tuned LLMs

The current model is a general multilingual LLM. Future work may involve:

- Sanskrit fine-tuning
- Indic instruction tuning
- Sanskrit-specific response generation

10.3 Multilingual Query Support

The system can be extended to support:

- English queries
- Hindi queries
- multilingual retrieval pipelines

10.4 Agentic Retrieval

Future versions may include:

- multi-hop retrieval

- query decomposition
- agentic retrieval planning
- dynamic context selection

10.5 Better Sanskrit Morphological Processing

Additional NLP modules such as:

- sandhi splitting
- morphological normalization
- Sanskrit POS tagging

can further improve retrieval quality.

---

11. Conclusion

This project successfully demonstrates a CPU-optimized Sanskrit Retrieval-Augmented Generation system capable of processing Sanskrit documents and answering Sanskrit queries using hybrid retrieval and local LLM inference.

The system combines:

- semantic retrieval
- keyword retrieval
- transliteration-aware processing
- quantized CPU-based generation

to create a practical and modular Sanskrit question-answering framework.

The implemented hybrid retrieval strategy significantly improves Sanskrit retrieval quality by combining semantic understanding with exact keyword matching.

The transliteration-aware interface further improves usability for Indic-language users by supporting Sanskrit queries written in Latin script.

Overall, the project demonstrates the feasibility of lightweight, retrieval-grounded Sanskrit AI systems and provides a strong foundation for future Bharatiya language AI applications.